The Journal of Open Source Software

AMIS: Python Package for Adaptive Multi-Interval Scale Normalization

Gennadiy Grigorievich Kravtsov [1]

1 Research Center "Applied Statistics"

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 14 December 2025
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

AMIS: PYTHON PACKAGE FOR ADAPTIVE MULTI-INTERVAL NORMALIZATION AND MAPPING OF HETEROGENEOUS DATA

Author: Gennadiy Grigorievich Kravtsov
Affiliation: Research Center "Applied Statistics"
ORCID: 0009-0000-3405-1461

Summary

**AMIS (Adaptive Multi-Interval Scale) is a normalization and mapping method that transforms heterogeneous metrics into a unified 0-100 interval scale. Unlike standard percentiles and z-scores, AMIS accounts for data distribution, preserves interval scale properties, and maintains label correspondence. The Python package implements five normalization models (3, 5, 9, 17 points, and linear), provides inverse transformation, and offers interactive graphical visualization. Version 4.3 introduces adaptive model selection that automatically adjusts available models based on data volume.

Statement of Need

Comparative analysis in interdisciplinary research is often limited by methodological incomparability of data. Traditional normalization methods have limitations: linear scaling ignores distribution; z-standardization requires normality; percentiles violate interval scale properties. Existing Python packages like scikit-learn do not fully address semantic comparability issues. **AMIS** creates a unified metric space suitable for normalizing educational grades, economic indicators, and sports performance, enabling meaningful cross-disciplinary comparisons.

Real-world applications: - **Dataset size adaptation:** Works reliably with varying data volumes (10-1000+ observations) - **Education:** Comparing student performance across different subjects and grading systems - **Economics:** Normalizing GDP, inflation, and employment metrics - **Healthcare:** Combining clinical scores with laboratory measurements - **Sports:** Standardizing athlete performance across different events

Existing Python packages such as scikit-learn provide normalization methods (`MinMaxScaler`, `StandardScaler`) but lack subject-specific adaptation. `MinMaxScaler` performs linear scaling that ignores data distribution, while `StandardScaler` assumes normality and produces unbounded results. The AMIS package addresses these limitations by providing distribution-aware normalization with fixed 0-100 boundaries, which is particularly important for educational and economic data where scale semantics must be preserved.

Implementation

The AMIS package implements a three-layer architecture separating core algorithms, user interface, and utility functions: **Core Algorithms (`amis_tool/core/amis_calculations.py`)** The `amis_safe_conversion()` function implements the adaptive multi-interval algorithm with these key features: - Hierarchical control point calculation through recursive mean computation (3, 5, 9, 17-point models) - Robust handling of edge cases: NaN filtering, duplicate point removal, fallbacks for empty intervals
- Automatic boundary determination from data (no fixed bounds required) - Simultaneous return of five normalization models for comparative analysis

The `load_data_to_array()` function loads data from DataFrames: - Extracts labels from the first column - Converts numerical values from the second column - Returns values, column name, and labels for downstream processing

Adaptive Model Selection (New in Version 4.3) AMIS v4.3 implements an adaptive model selection system that automatically determines available normalization models based on data volume:

Data Points	Available Models	Statistical Justification
10-19	Linear + 3-point	Minimum for stable mean estimates
20-49	+ 5-point	Sufficient data for 5 intervals
50-99	+ 9-point	Adequate for high-precision normalization
100+	All 5 models	Full hierarchical averaging possible

This adaptive hierarchy ensures: - **Statistical reliability:** Prevents overfitting with small datasets - **Automatic UI adaptation:** Interface disables unavailable models - **Progressive enhancement:** More data enables more precise models - **Error prevention:** Minimum 10-point requirement validated at load time

Graphical Interface (`amis_tool/gui/`) The Tkinter-based application provides interactive access for non-programmers: - `main_window.py`: Main application window with data loading, model selection, and export functions - `dialogs.py`: Specialized dialogs for data comparison settings
- `widgets.py`: Custom table viewer displaying original and normalized values side-by-side

Utility Functions (`amis_tool/utils/helpers.py`) Helper functions for window management and user experience enhancement.

The entry point `amis_tool/__main__.py` initializes the application with a splash screen and launches the main interface shown in Figure 1.

Statistical Requirements

AMIS requires a minimum of 10 observations for reliable operation. Version 4.3 introduces **adaptive model selection** that determines available normalization models based on data volume:

- **10-19 observations:** Linear + 3-point models (basic normalization)
- **20-49 observations:** Additional 5-point model (enhanced precision)

- **50-99 observations:** Additional 9-point model (high precision)
- **100+ observations:** All 5 models available (maximum precision)

This adaptive approach ensures statistical validity while providing progressive enhancement as data volume increases, addressing a common limitation in normalization tools that either fail with small datasets or produce unreliable results.

```
def get_available_models(n):
    """Determine available models based on data volume"""
```

DRAFT

```
    if n >= 100:
        return {"17_points": "17 points", "9_points": "9 points",
                "5_points": "5 points", "3_points": "3 points",
                "linear": "Linear"}
    elif n >= 50:
        return {"9_points": "9 points", "5_points": "5 points",
                "3_points": "3 points", "linear": "Linear"}
    elif n >= 20:
        return {"5_points": "5 points", "3_points": "3 points",
                "linear": "Linear"}
    elif n >= 10:
        return {"3_points": "3 points", "linear": "Linear"}
    else:
        raise ValueError("Minimum 10 points required")
```

Design Decisions - **Minimum samples:** 10 observations required for statistically stable mean estimation - **Visualization:** Four-panel matplotlib comparison (Figure 2) showing distribution preservation - **Compatibility:** CSV/Excel input support with pandas integration - **Label preservation:** Original identifiers maintained in normalized output tables

Figure 1: AMIS Normalization Interface

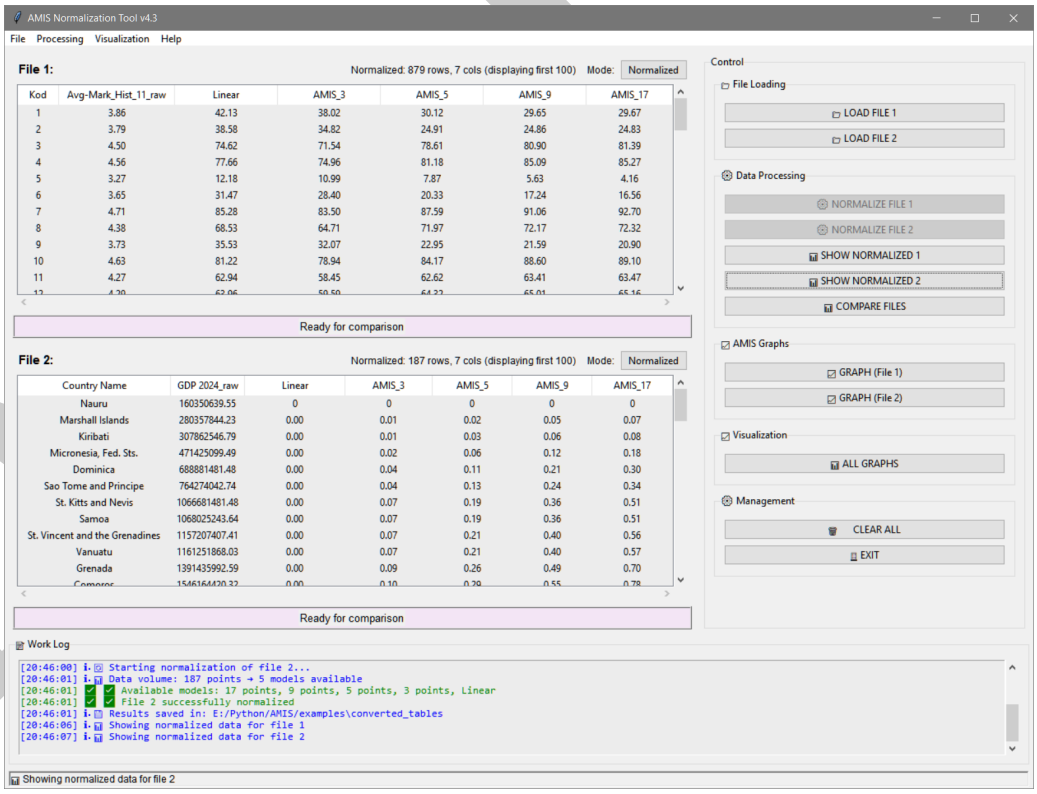

Figure 1: AMIS Normalization Interface

Figure 1: AMIS graphical interface simultaneously normalizing heterogeneous educational and economic data: history grades from Russian schools (`Student_Grades_History_Grade11_Raw_Data.xlsx`, left panel) and country GDP values (`World_Bank_Nominal_GDP_All_Countries_2024.xlsx`, right panel).

The Journal of Open Source Software

Figure 2: Multi-panel visualization of AMIS normalization and cross-dataset mapping

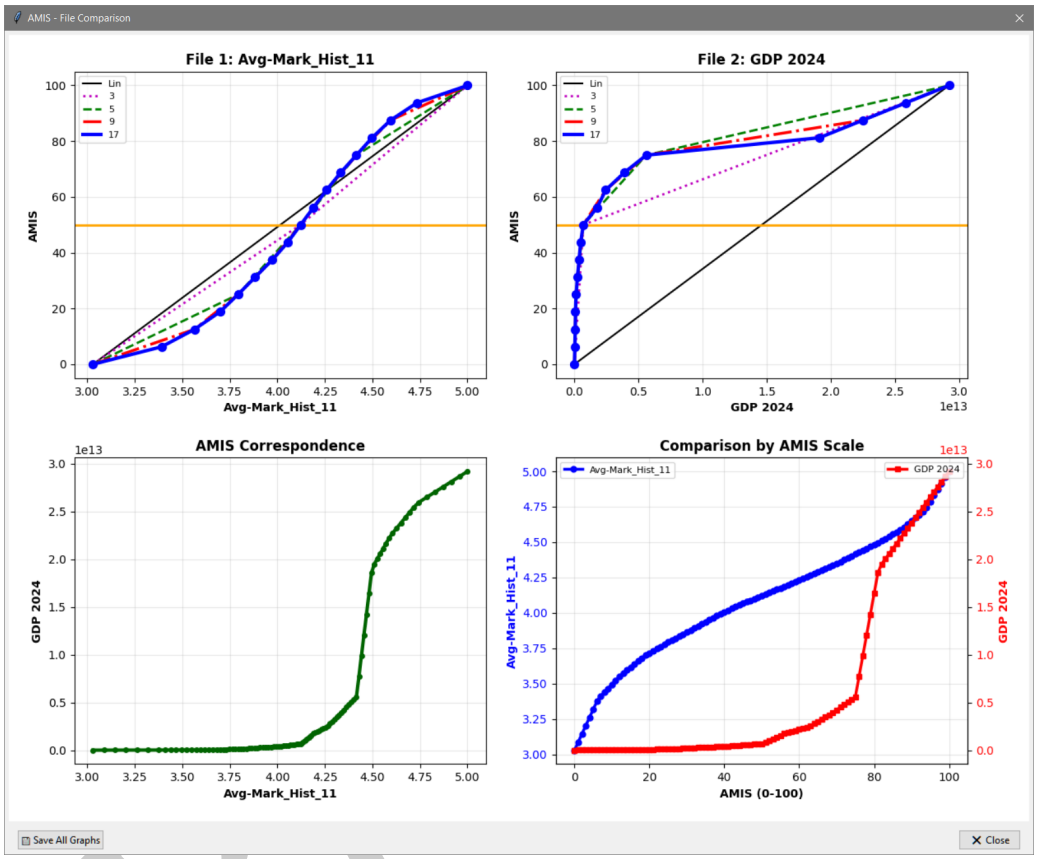

Figure 2: AMIS Normalization Analysis

Figure 2: Four-panel analysis of cross-metric normalization:
- **Top-left:** AMIS curve mapping history grades (3.03-5.00) to 0-100 AMIS scale - **Top-right:** AMIS curve mapping country GDP (1.6×10 -2.9×10^{13} USD) to 0-100 AMIS scale
- **Bottom-left:** Correspondence between educational and economic AMIS scales
- **Bottom-right:** Unified comparison showing relative positions on shared 0-100 axis

Examples

Basic Normalization with Real Educational Data

```
import pandas as pd
from amis_tool.core.amis_calculations import load_data_to_array, amis_safe_conversion

# Load real historical grade dataset (879 teacher average scores)
# Data: History subject teacher averages, 11th grade, Russian schools
df = pd.read_excel('Student_Grades_History_Grade11_Raw_Data.xlsx')

# Extract numerical values, column name, and teacher/school identifiers
values, column_name, labels = load_data_to_array(df)

print(f"Dataset: {len(values)} teacher average scores for {column_name}")
print(f"Data range (auto-detected): {values.min():.2f} to {values.max():.2f}")
```

```python
# Apply AMIS normalization - returns all 5 models simultaneously
normalized, points = amis_safe_conversion(values)

print(f"\nAMIS provides 5 normalization models (example for first record):")
print(f"  Teacher {labels[0]}: {values[0]:.2f} original score")
print(f"    • 3-point model: {normalized['3_points'][0]:.1f} AMIS")
print(f"    • 5-point model: {normalized['5_points'][0]:.1f} AMIS")
print(f"    • 9-point model: {normalized['9_points'][0]:.1f} AMIS")
print(f"    • 17-point model: {normalized['17_points'][0]:.1f} AMIS")
print(f"    • Linear scaling: {normalized['linear'][0]:.1f} AMIS")

print(f"\nControl points for 5-point model (original → AMIS):")
for x, y in zip(points['x_5'], points['y_5']):
    print(f"  {x:.2f} → {y:.1f}")

print(f"\nFirst 3 records across all 5 models:")
models = ['3_points', '5_points', '9_points', '17_points', 'linear']
model_labels = ['3-pt', '5-pt', '9-pt', '17-pt', 'linear']

for i in range(3):
    results = [f"{label}:{normalized[model][i]:.1f}"
               for label, model in zip(model_labels, models)]
    print(f"  {labels[i]}: {values[i]:.2f} → {', '.join(results)}")
```

Complete working examples are provided in the repository's `/examples` directory. For interactive analysis with the graphical interface, run `python -m amis_tool`.

Availability

Source code: https://github.com/Famimot/AMIS
Test datasets: https://doi.org/10.7910/DVN/HXSED6
Preprint: https://doi.org/10.5281/zenodo.17816551
License: MIT (see LICENSE file in repository)
Requirements: Python 3.8+ with pandas, numpy, scipy, matplotlib

References

1. Kravtsov, G. G. (2025). Universal Adaptive Normalization Scale (AMIS): A methodology for integrating heterogeneous social and educational metrics. *OSF Preprints*. https://doi.org/10.17605/OSF.IO/BDT2K

DRAFT